

Ex. No.: 7

Experiment Title: IPC Using Shared Memory

Aim:

To write a C program to do Inter Process Communication (IPC) using shared memory between a sender process and a receiver process.

Algorithm:

Sender:

1. Set the size of the shared memory segment.
2. Allocate the shared memory using `shmget()`.
3. Attach it to the address space using `shmat()`.
4. Write a message to shared memory using `sprintf()`.
5. Use `sleep()` to pause for observation.
6. Detach the shared memory using `shmdt()`.

Receiver:

1. Set the size of the shared memory segment.
2. Access the shared memory using `shmget()`.
3. Attach it to the address space using `shmat()`.
4. Read and display the message from shared memory.
5. Detach the shared memory using `shmdt()`.

Program Code – sender.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>
#include <string.h>

int main() {
    key_t key = ftok("shmfile",65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char*) shmat(shmid, (void*)0, 0);

    sprintf(str, "Welcome to Shared Memory");
    printf("Message Sent: %s\n", str);

    sleep(5); // Optional delay to keep shared memory alive for
receiver

    shmdt(str);
    return 0;
}
```

Program Code – receiver.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

int main() {
    key_t key = ftok("shmfile",65);
    int shmid = shmget(key, 1024, 0666);
    char *str = (char*) shmat(shmid, (void*)0, 0);

    printf("Message Received: %s\n", str);
}
```

```
    shmdt(str);  
    return 0;  
}
```

Sample Output:

Terminal 1:

```
[root@localhost student]# gcc sender.c -o sender  
[root@localhost student]# ./sender  
Message Sent: Welcome to Shared Memory
```

Terminal 2:

```
[root@localhost student]# gcc receiver.c -o receiver  
[root@localhost student]# ./receiver  
Message Received: Welcome to Shared Memory
```

Result:

The programs successfully demonstrated IPC using shared memory, with the sender writing a message that the receiver was able to access and display.